

# こども プログラミング教室

ア ド バ ン ス コ ー ス

## *Advance course* SU-2

### *Start Up*

授業日

年 月 日

名 前



**human**  
ヒューマンアカデミー ジュニア  
STEAMスクール



こども  
プログラミング教室

## Start Up 2回目 もくじ

## ① JavaScript と Scratch のちがい

P1

めやす  
20ふん

- 1 JavaScript の考え方 ～行動を分けて考える …… P1～P2
- 2 JavaScript と Scratch で似ているところ …… P3～P4
- 3 JavaScript と Scratch でちがうところ …… P5
- 4 JavaScript と Scratch で同じ意味のプログラムを、  
見比べてみよう …… P6

## ② ジャンケンゲームを作ろう

P7

めやす  
60ふん

- 1 ゲームルールの説明 …… P7
- 2 ジャンケンゲームの画面の流れ …… P8
- 3 プログラミングを始める前に  
～ JavaScript のプログラムの作り～ …… P9 ～ P11
- 4 タイトル画面を作ろう① …… P12～P14
- 5 タイトル画面を作ろう② …… P15
- 6 タイトル画面を表示しよう …… P16
- 7 プレイ画面を表示しよう …… P17～P18
- 8 結果画面を表示しよう …… P19
- 9 最終結果画面を表示しよう …… P20

## ③ 改造しよう

P21

めやす  
10ふん

- 1 5回勝負にしてみよう …… P21～P22
- 2 コンピュータを強くしてみよう …… P23～P24

## 1

ジャバ スクリプト スクラッチ

## JavaScript と Scratch のちがい

1 回目の授業では JavaScript の説明やコード入力の仕方を学び、タイピング練習を行いました。が、今回は JavaScript で使用される基本的なコマンド（コード）を学んだり、1 回目に出てきた「ジャンケン」の画ぞうを使って「ジャンケンゲーム」を作ったりします。

まずは JavaScript のプログラムの基本的な構造、組み立て方を知っておきましょう。

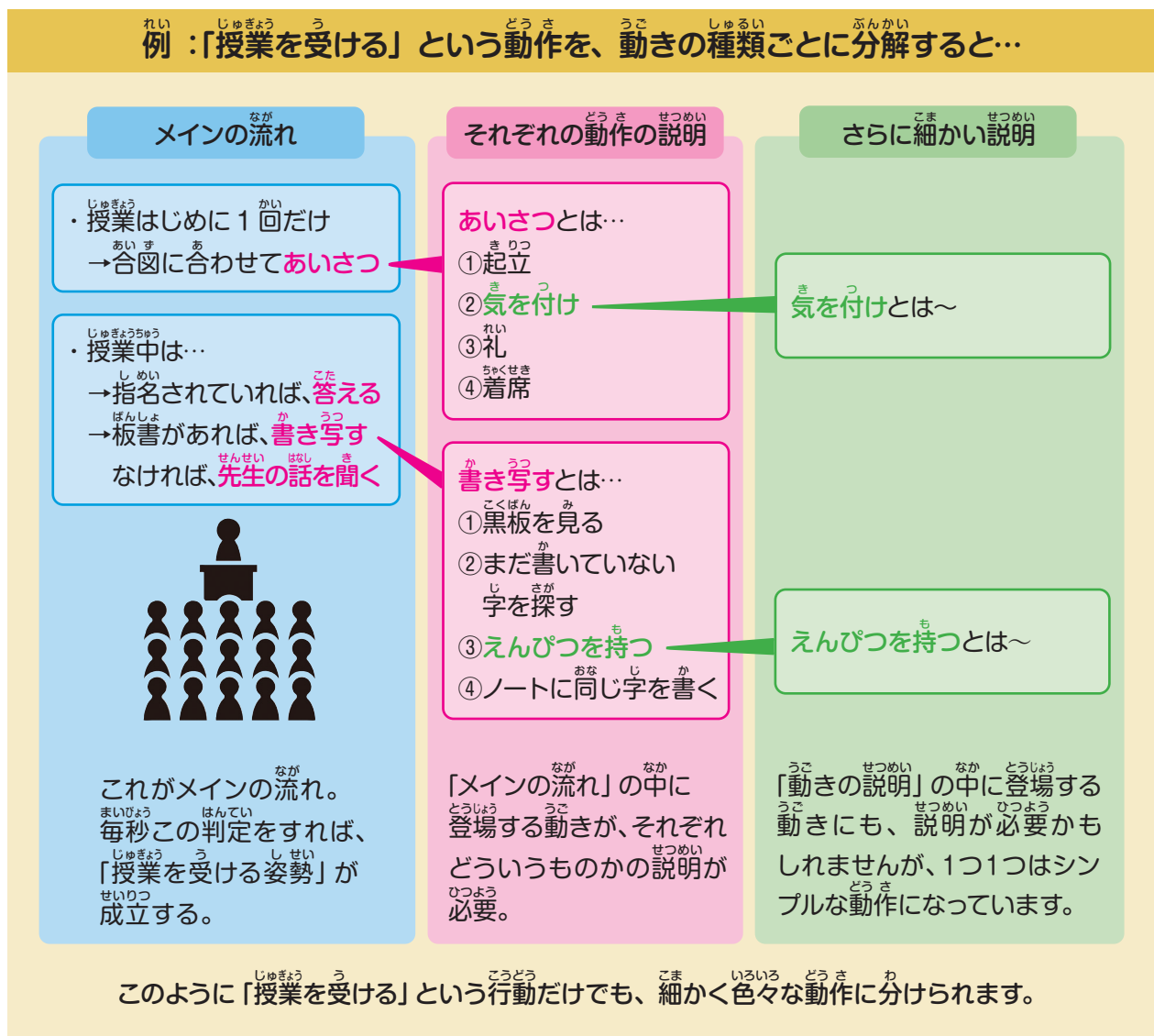
## 1

## JavaScript の考え方 ～行動を分けて考える

JavaScript のプログラムの作りについて理解を深めるため、たとえ話として「授業を受ける」という動作を、JavaScript 風に記さいてみましょう。

授業を受ける動作を細かく分けてみると、以下のようになります。

例：「授業を受ける」という動作を、動きの種類ごとに分解すると…



では、これらの動作をJavaScript風に書きかえてみましょう。

JavaScriptでは動作1つ1つを「function」というカタマリとしてグループ分けします。

それぞれのfunctionには、基本的には自由に名前を付けることができます。

JavaScriptでは、一つ一つのカタマリを「function」と呼びます。

```
function start() {
  if (状況 === 授業はじめ){
  }
}
```

あいさつ()

```
function あいさつ() {
  起立()
  気を付け()
  礼()
  着席()
}
```

```
function 気を付け() {
  ~~
}
```

```
function mainloop() {
  if (指名されている){
    答える()
  }

  if (板書 === あり){
    書き写す()
  }

  else {
    先生の話を聞く()
  }
}
```

答える()

書き写す()

先生の話を聞く()

```
function 書き写す() {
  黒板を見る()
  書いていない字を探す()
  ペンを持つ()
  ノートに書く()
}
```

```
function ペンを持つ() {
  ~~
}
```

これらを全てまとめると、1つの「プログラム」になる。

基本的な命令は、他でも使いまわせるので別ファイルにまとめる。("parts.js" など)

・細かくしよ理を分けることで、どのカタマリがどの行動を担当しているのかわかりやすくなる！

・カタマリでまとめるとそのカタマリをくり返し使うことができるので、同じ動作が何回も行われるようなプログラムの場合、細かい命令を何回も書く必要がない！

・エラーが起きたとき、「どの動きでエラーが起きているか」が分かれば、間ちがっている部分を見つけやすい！

プログラムが長く複雑になってきても、ひとまず function ごとに区切って読んでいけば、どこに何の命令が書かれているかわかりやすくなりますね！

実際に授業で使う function の名前の例

<script ...> プログラムを書く部分

```
function init() {
  最初に1回だけやること (起動しより)
}
```

```
function start() {
  ゲームの開始の時にやること (開始しより)
}
```

```
function keyUp(evt) {
  キーをおし終えた時にやること
}
```

```
function keyDown(evt) {
  キーがおされた時にやること
}
```

```
function mainloop() {
  一定時間おきにやること
}
```

```
function draw() {
  画面をえがく
}
```

</script>

## 2

## JavaScript と Scratch で似ているところ

次に、JavaScript の命令の書き方を、今までやってきた Scratch のブロックの命令とくらべて確認してみましょう。

書き方がちがうだけの  
ところがたくさんあるよ！



## 1 演算

演算とは足し算やかけ算のような数字の計算のことです。

四則計算（＋－×÷）の表し方は、ほとんど変わりません。

四捨五入やあまりの数の計算などは、Scratch とは少し違う書き方で命令を出す必要があります。

## 四則演算

| Scratch | JavaScript  |
|---------|-------------|
|         | 1 + 2       |
|         | 1 - 2       |
|         | 1 * 2       |
|         | 1 / 2       |
|         | (1 + 3) / 2 |

## その他の計算

| Scratch | JavaScript      |
|---------|-----------------|
|         | 7 % 3           |
|         | Math.round(1.5) |
|         | Math.abs(-2)    |
|         | Math.floor(1.2) |
|         | Math.ceil(1.2)  |
|         | Math.sqrt(4)    |

## 2 数値の比かく

「もし」の条件などに使われる、比かくの命令です。記号はほとんど変わりません。

「かつ」「または」など言葉で表されていた命令も、JavaScript では記号で表現します。

## 数値の比かく

| Scratch | JavaScript                         |
|---------|------------------------------------|
|         | memberNum < 50                     |
|         | memberNum === 50                   |
|         | memberNum > 50                     |
|         | memberNum > 50    memberNum === 50 |

## かつ・または・否定

| Scratch | JavaScript                         |
|---------|------------------------------------|
|         | memberNum > 50 && word === "hello" |
|         | memberNum > 50    word === "hello" |
|         | !(word === "hello")                |

### 3 せい 制ぎよ

「もし」や「くりかえす」の命令は、JavaScript でも存在しています。  
 かつこ ( ) の中に条件、波かつこ { } の中に動きの命令を入れます。

#### もし…なら

| スクラッチ<br>Scratch | JavaScript                                                                          |
|------------------|-------------------------------------------------------------------------------------|
|                  | <pre>if (word === "hello") {   // 条件が真だった場合の処理 } else {   // 条件が真でなかった場合の処理 }</pre> |

#### くり返し

| スクラッチ<br>Scratch | JavaScript                                                                                                         |
|------------------|--------------------------------------------------------------------------------------------------------------------|
|                  | <pre>for (let step = 0; step &lt; 10; step++) {   // step が 0 から 9 まで一つずつ増える間、   // 合計 10 回 {} 内の処理が繰り返される }</pre> |

### 4 コード全体を比べてみよう

ここまでしょうかいした命令たちは、Scratch でも同じような意味のブロックがありましたね。  
 Scratch では「命令が書かれたブロックを置いて、数字を変えていく」という場面が多かったですが、  
 JavaScript のプログラミングも、「命令の文から自分で書く必要がある」ということ以外は、Scratch  
 のプログラミングと考え方は似ています。

| スクラッチ<br>Scratch | JavaScript                                                                                                                 |
|------------------|----------------------------------------------------------------------------------------------------------------------------|
|                  | <pre>let memberNum = 20 if (memberNum &gt; 50) {   word = "50人より多いよ" } else {   word = "50人以下だよ" } console.log(word)</pre> |

かんが かんが  
考え方

変数 memberNum を 20 にする  
 もし memberNum が 50 より多ければ  
 変数 word を "50 人より多いよ" にする  
 そうでなければ  
 変数 word を "50 人以下だよ" にする  
 変数 word を表示する



## 3

JavaScript スクラッチ

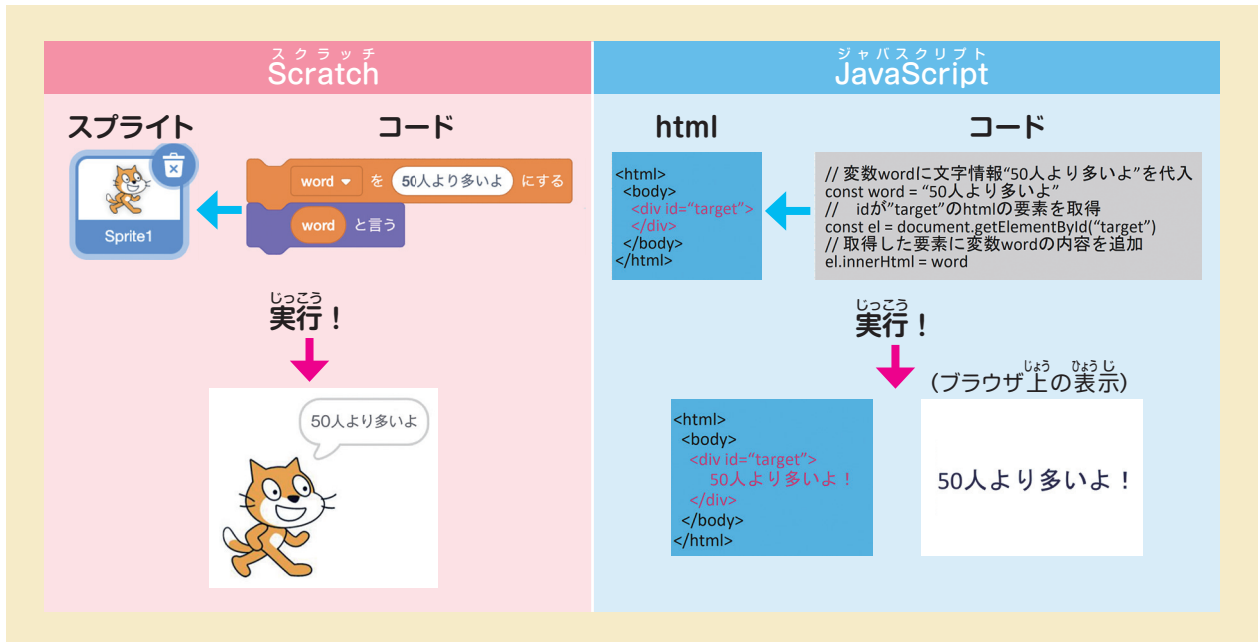
## JavaScript と Scratch でちがうところ

## 1 キャラクターごとにプログラムを分けない！

きをつけていこう！



スクラッチはキャラ（スプライト）ごとにプログラムを作り分けて、「だれが実行する命令か」を指示していましたが、JavaScriptの場合、1つのプログラムの中で「この命令は、どこで／どのキャラが実行するか」を指示することになります。



## 2 タイピングのミスでエラーが起こる！

Scratchは、命令の内容が初めからブロックに書かれていたので、それを使うだけでよかったのですが、JavaScriptは、自分で命令文を打ち込んでいくことになります。よって、命令文を打ちまちがえたりすると「そもそもプログラムを実行できない」というエラーが起こることがあります。たとえば、命令の言葉をまちがえたり、必要な記号（カッコなど）をわすれたりすることによるエラーが多いです。

## JavaScript のミスの例

## タイプミス

```
> // 変数wordに文字情報"50人より多いよ"を代入
const word = "50人より多いよ"
// idが"target"のhtmlの要素を取得
const el = document.getElementById("target")
// 取得した要素に変数wordの内容を追加しようとして、wardを追加(動かない)
el.innerHTML = ward
```

✖ Uncaught SyntaxError: Invalid or unexpected token

英語のつづり間ちがいです。上記の例では、“word”と書くべきところが“ward”になってしまっています。

## シンタックスエラー

```
> let memberNum = 20
if (memberNum > 50) {
  word = "50人より多いよ"
} else {
  word = "50人以下だよ"
}
// <- "}"で閉じるところを")"にミス
console.log(word)
```

✖ Uncaught SyntaxError: Unexpected token ')'

プログラミングのシンタックス（書き方ルール）に間ちがいがある場合です。上記の例では、カッコの数が足りていません。

## 4

## JavaScript と Scratch で同じ意味のプログラムを、見比べてみよう

下の2つのプログラムは、どちらも同じしよ理を表しています。

書き方がちがうところと同じところを下にまとめているので、見比べてみましょう。

| スクラッチ<br>Scratch | JavaScript                                                                                                                                                                                                                                                                        |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | <pre>&lt;script type = "text/javascript"&gt;   let hensu = 0;   for(let i = 0; i &lt; 10; i++){     hensu = hensu + 1 + Math.floor(Math.random() * 10);   }   if( hensu &lt; 50){     document.write("こんにちは!");   }else {     document.write("こんばんわ!");   } &lt;/script&gt;</pre> |

### 緑の旗がおされたとき

JavaScript には「旗がおされたとき」と同じような命令はありません。

html を開くことや、こう新すること自体が同じような意味になっています。

### 10 回くり返す

Scratch と違い JavaScript は基本的に 0 が始まりになることがほとんどです。くり返しても 0 から始めるため、10 回くり返したい場合は「10 より小さい整数のあいだくり返す」とする必要があります。

### もし～なら

JavaScript と Scratch ではほとんどちがいはありません。

今までの考え方で作って問題ありません。

なるほど





## 2

## ジャンケンゲームを作ろう

## 1

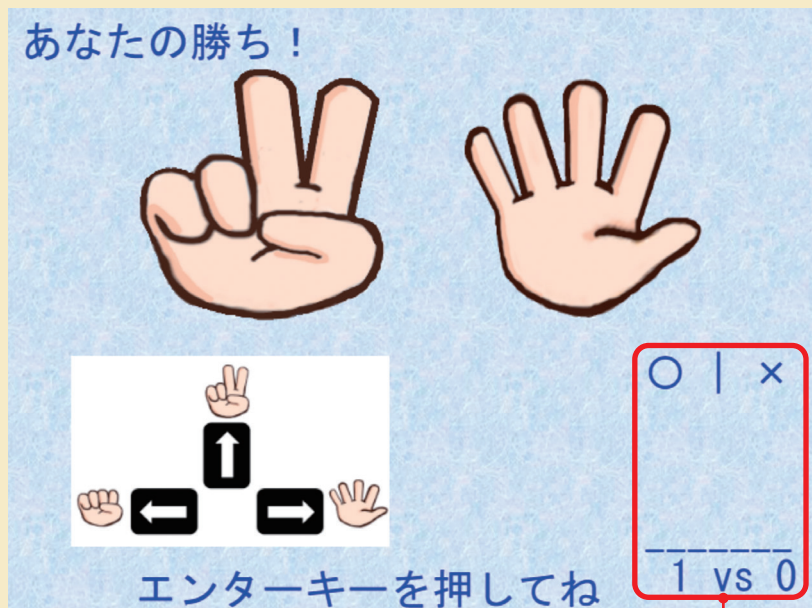
## ゲームルールの説明

コンピュータと、ジャンケンの3回勝負をするゲームを作成します。コンピュータが出す「グー」「チョキ」「パー」に対して、自分が出す手を矢印キーで決定して勝負を行います。あいこの場合はやりなおし。ジャンケン勝負のプログラミングを通じて、JavaScript プログラミングの基本を学んでいきましょう。

こんなゲームを作るよ！

ジャンケンでコンピュータと3回勝負！

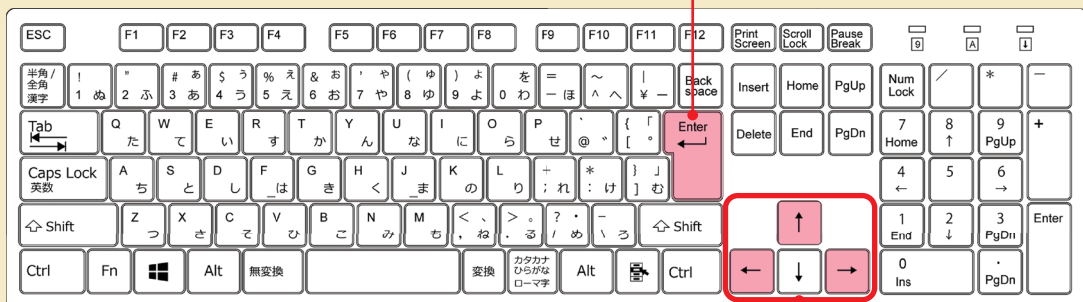
あなたの勝ち！



エンターキーを押してね

③対戦結果が表示されます。

①エンターキーで  
ゲームスタート



②矢印キーで  
自分の手を選びます。

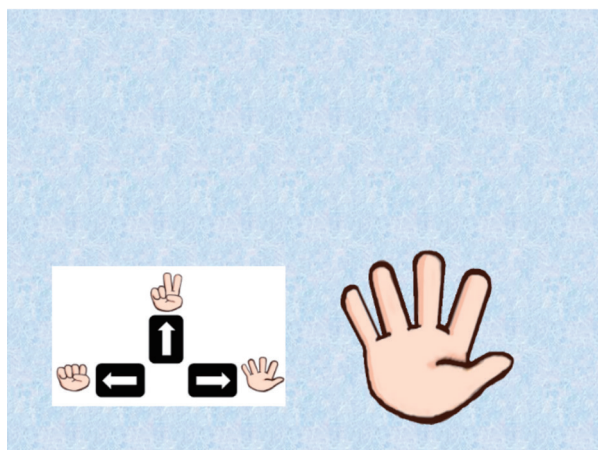
## 2

## ジャンケンゲームの画面の流れ



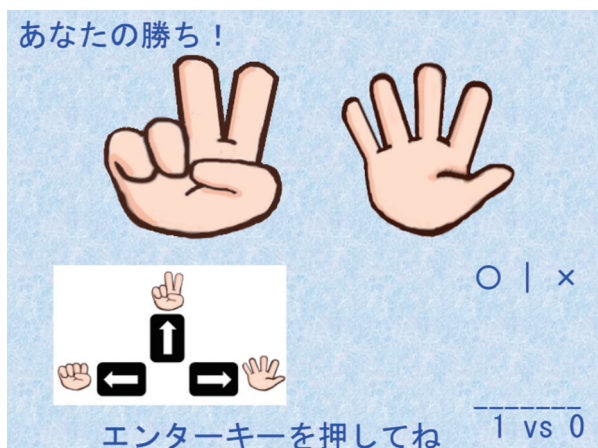
## ①「タイトル画面」

エンターキーをおすとゲーム開始です。



## ②「プレイ画面」

コンピュータの手が「グー」→「チョキ」→「パー」とぐるぐる回りながら表示されるので、矢印キーで自分の手を選びます。



## ③「結果画面」

3回勝負の途中結果が表示されていきます。



## ④「最終結果画面」

3回勝負の最終結果が表示されます。勝つのはどちらかな？

## 3

## プログラミングを始める前に ~ JavaScript のプログラムの作り~

JavaScript ではプログラムを細かい「function」というカタマリに分けます。  
 ジャンケンゲームも、手順をいくつかのカタマリに分けて作って行きます。

## JavaScript のプログラムの一般的な作り

```
<script ...> プログラムを書く部分
function keyUp(evt) {
  キーをおし終えた時にやること
}

function keyDown(evt) {
  キーがおされた時にやること
}

function mainloop() {
  一定時間おきにやること
}

function draw() {
  画面をえがく
}
</script>
```

エンターキーで  
ゲームが始まることを、ここに書きます。

矢印キーで自分の手を決めるときの  
命令を、ここに書きます。

タイトル画面を表示することを、  
ここに書きます。

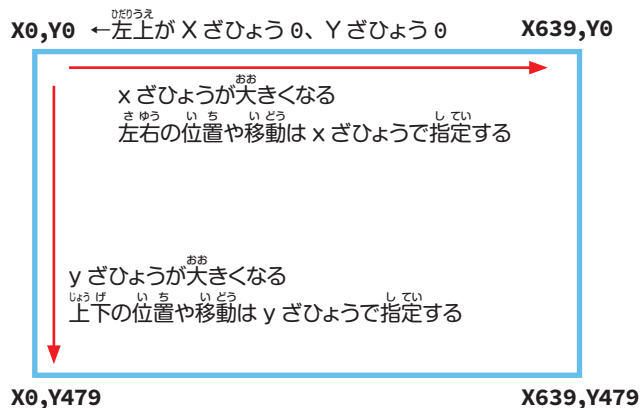


## function とは

いくつかの命令（しよ理）を1つに  
まとめて、付けられた名前呼び出し  
て使えるようにするものです。

## x ざひょう、y ざひょうの位置について

JavaScript では、左上が x ざひょう 0、  
y ざひょう 0 となります。



## Scratch の時とは、

ざひょうのはいちがちがうので  
注意しよう。

◎ 左上がスタートポイント  
※ Scratch では真ん中だったね。

◎ y ざひょうは、下に行けば行くほど  
数値が大きくなる。  
※ Scratch では下に行くと数値が  
小さくなったけど、JavaScript  
では大きくなるよ。

アドバンスコースのプログラミングは TeraPad で行っていますが、今回作る「ジャンケンゲーム」については、入力結果の確認のために、1 回目の授業でタイピングの練習に使った〈コード写経〉を使って入力してみましょう。

〈コード写経〉の使い方は、下記を参照してください。

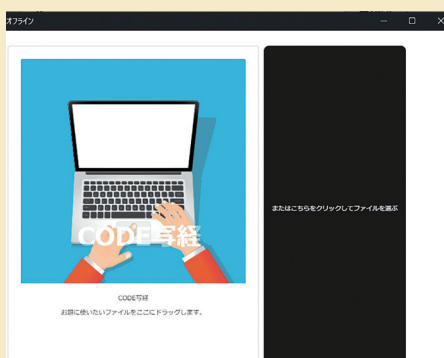
## 〈コード写経〉の使い方

### ① 〈コード写経〉を起動する

パソコンのデスクトップ画面にある〈コード写経〉のアイコンをダブルクリックし、〈コード写経〉アプリを起動します。

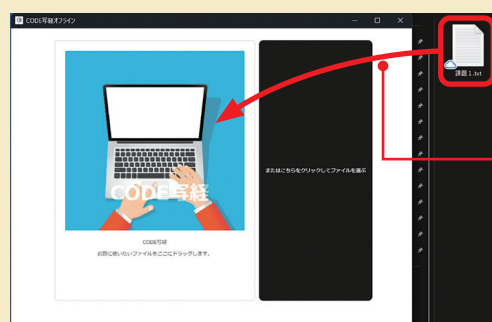


アイコンを  
ダブルクリック

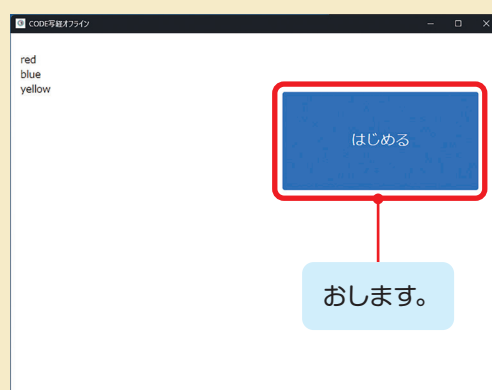


### ② 〈コード写経〉に課題を読みこむ

デスクトップに保存されている課題ファイルを、〈コード写経〉の画面の左側にドラッグ & ドロップします。



ドラッグ&ドロップ  
します。

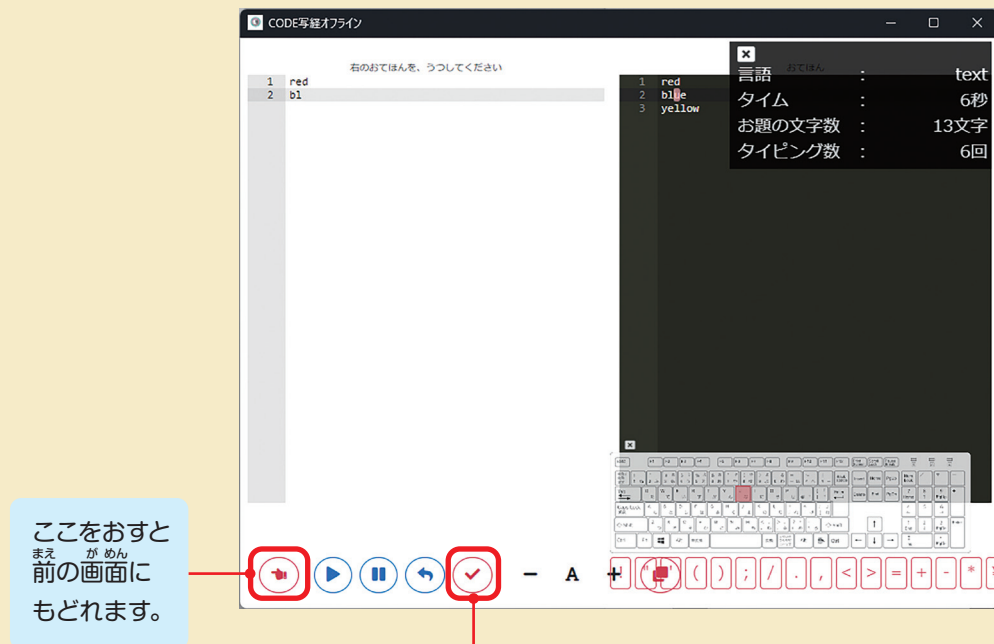


おします。

準備ができたなら、  
「はじめる」をおします。

### ③ 〈コード写経〉でタイピング練習する

右画面に表示されているお手本課題をみながら、左画面にタイピングしていきます。

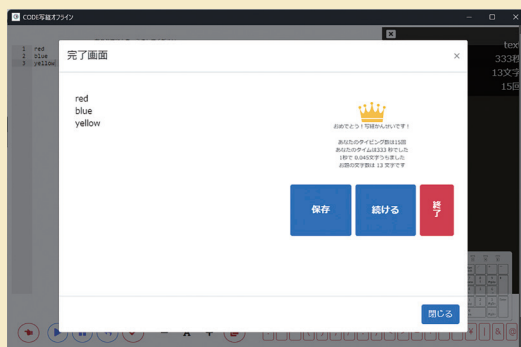


ここをおすと  
まえ  
前の画面に  
もどれます。

### ④ 入力結果を確認する

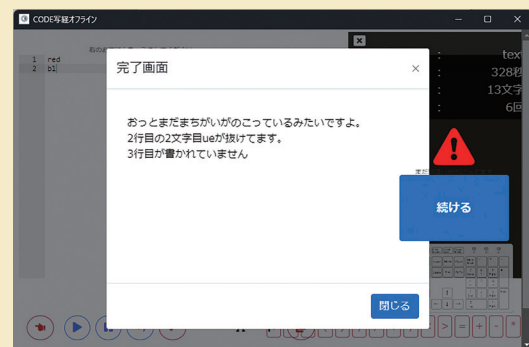
全部の文字をタイピングできたら完成です。✓をクリックして間ちがいがないか、結果を確認しましょう。

タイピング成功の場合



タイピング成功

間ちがいが残っている場合



どこかに間ちがいが残っている

間ちがいが残っている場合には、右の画面のようにどこが間ちがえているかが表示されます。「続ける」をおして、表示されたところを確認してみましょう。



## 4

## タイトル画面を作ろう①

まず、ゲームの開始画面を作りましょう。

下記のようなデザインにするには、どうしたらよいでしょうか。

ひょうじ が めん  
表示画面

まず、グーを1つ表示するところから始めます。



## 課題1 グーを1つ表示しよう

画面に絵を出す命令を書いてみましょう

## 入力するテキスト

```
// 画面をえがく
function draw() {
  ctx.drawImage(gu, 150, 100, 100, 100)
}
```

ひょうじ  
表示する  
が めん  
画像ファイル名

ひょうじ いち  
表示する位置  
(x ざひょう, y ざひょう)

ひょうじ  
表示する  
おお  
大きさ

## プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_ お手本\_ ステップ\_4-1.html]** を読みこみます。
- ② 〈コード写経〉のプログラミングエリアに **[index\_ 課題.html]** を読みこみます。
- ③ コードの赤字の部分をタイピングして完成後にコードを保存します。
- ④ 保存したファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

画面にグーが1つ表示されたかな？

☐






## グーを2つ表示しよう

つぎに、グーを2つ、タテに表示しましょう。“くり返し”は、「for 文」を使って書きます。

表示画面



入力するテキスト

```
// 画面をえがく
function draw() {
  for (let i = 0; i < 2; i++) {
    ctx.drawImage(gu, 150, 100 + 120 * i, 100, 100)
  }
}
```

$i < 2$  は、次に書いてある「i++」を2回くり返す、という意味。

i++ は、変数 i の値を1ずつ増やす、という意味。

表示するyざひょうの位置を  $120 \times i$  の数の分だけ下に下ろしていく、という意味。



### “くり返し”の命令

スクラッチでは右記のブロックを使っていたが、JavaScriptでは「for」を使います。



## プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_4-2.html]** を読みこみます。
- ② コードの赤字の部分のタイピングして完成後にコードを保存します。
- ③ 保存したファイルをChromeブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

画面にグーが2つ、タテに表示されたかな？

☐




## グーを3つ表示しよう

くり返しの回数を書きかえるだけで、グーを3つ表示できます。

表示画面



入力するテキスト

```
// 画面をえがく
function draw() {
  for (let i = 0; i < 3; i++) {
    ctx.drawImage(gu, 150, 100 + 120 * i, 100, 100)
  }
}
```

### プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_4-3.html]** を読みこみます。
- ② コードの赤字の部分の部分をタイピングして完成後にコードを保存します。
- ③ 保存したファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

3回くり返すときも、プログラムの長さは変わらないんだね!



画面にグーが3つ、タテに表示されたかな?

☐


## 5 タイトル画面を作ろう②



### グーチョキパーを3列表示しよう

同じやり方で、チョキとパーの画像をそれぞれタテに3つつ追加します。

#### 入力するテキスト

```
function draw() {
  for (let i = 0; i < 3; i++) {
    ctx.drawImage(gu, 150, 100 + 120 * i, 100, 100)
    ctx.drawImage(choki, 250, 100 + 120 * i, 100, 100)
    ctx.drawImage(pa, 350, 100 + 120 * i, 100, 100)
  }
}
```

#### プログラミングの手順

- ①〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_5-1.html]** を読みこみます。
- ②コードの赤字の部分をタイピングして完成後にコードを保存します。
- ③保存したファイルを Chrome ブラウザにドラッグ&ドロップします。  
⇒プログラムが実行されます。

#### 表示画面



### 「if」(もし) を使ってチョキは2行目だけに表示しよう

#### 入力するテキスト

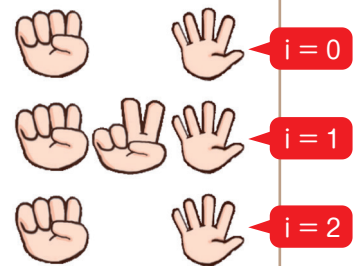
```
// 画面をえがく
function draw() {
  for (let i = 0; i < 3; i++) {
    ctx.drawImage(gu, 150, 100 + 120 * i, 100, 100)
    if (i === 1) {
      ctx.drawImage(choki, 250, 100 + 120 * i, 100, 100)
    }
    ctx.drawImage(pa, 350, 100 + 120 * i, 100, 100)
  }
}
```

チョキを表示する命令は、iが1の時だけ実行するようにしています。

#### プログラミングの手順

- ①〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_5-2.html]** を読みこみます。
- ②コードの赤字の部分をタイピングして完成後にコードを保存します。
- ③保存したファイルを Chrome ブラウザにドラッグ&ドロップします。  
⇒プログラムが実行されます。

#### 表示画面



#### “もし”の命令

スクラッチ  
Scratchでは右記のブロックを使っていたが、JavaScriptでは**if**を使います。



グーが3つ、チョキが1つ、パーが3つ表示されたかな？



## 6 タイトル画面を表示しよう

ひょうじ がめん  
表示画面



### スタート時にタイトル画面を表示しよう

仕上げに、ここまで学んだプログラムをくっつけて、画像とテキストを表示してみましょう。

にゅうりょく

#### 入力するテキスト

// 画面をえがく

function draw() {

if(mode === 0) {

ctx.drawImage(haikei, 0, 0, 640, 480)

for (let i = 0; i < 3; i++) {

ctx.drawImage(gu, 150, 100 + 120 \* i, 100, 100)

if (i === 1) {

ctx.drawImage(choki, 250, 100 + 120 \* i, 100, 100)

}

ctx.drawImage(pa, 350, 100 + 120 \* i, 100, 100)

}

ctx.fillStyle = 'blue'

ctx.font = '50pt MS ゴシック'

ctx.fillText('ジャンケンゲーム!', 50, 80)

ctx.font = '25pt MS ゴシック'

ctx.fillText('エンターキーをおしてね!', 100, 450)

}

}

がめんうえ ひょうじ  
画面上のタイトル表示

がめんか せつめいひょうじ  
画面下の説明表示

#### プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_6-1.html]** を読みこみます。
- ② コードの赤字の部分をタイピングして完成後にコードを保存します。
- ③ 保存したファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

グーチョキパーの画像とタイトル文字が表示されたかな？

☐


## 7

が めん ひょう じ  
プレイ画面を表示しよう

## エンターキーでゲームがスタートするようにしよう

エンターキーをおしたら、スタート画面からプレイ画面に切りかわるようにします。

ひょう じ が めん  
表示画面

プレイ画面はこのようになります。

にゅうりょく

## 入力するテキスト

```
// キーをおし終えた時にやること
function keyUp(evt) {
  if (mode === 0) {
    if (evt.keyCode === 13) {
      mode = 5
    }
  }
}
```

JavaScript

## JavaScript でキーボードのキーをおし終えた時のプログラムの書き方

```
// キーをおし終えた時にやること
function keyUp(evt) {
  if (evt.keyCode === 13) {
    // キーをおし終えた時の処理を記載する
  }
}
```

JavaScript では、キーボードのキーに数字が割り当てられていて、その数字を左記のプログラムの中を書くことで、そのキーをおし終えた時にやることを決めることができます。

この授業では、下記のキーコードを使います。

| Enter | ←  | ↑  | →  |
|-------|----|----|----|
| 13    | 37 | 38 | 39 |

つぎ  
次のページの手順に従って  
プログラミングを行ってください。







## プレイ画面のはい景とコンピュータの手を表示しよう

つぎに、プレイ画面に入った後の命令を書きます。

プレイ画面に入ったら、グーチョキパーが順番にくり返し表示されるようにしましょう。

にゅうりよく

### 入力するテキスト

```
// 画面をえがく
function draw() {
  if(mode === 0) {
    ctx.drawImage(haikei, 0, 0, 640, 480)
    for (let i = 0; i < 3; i++) {
      ctx.drawImage(gu, 150, 100 + 120 * i, 100, 100)
      if (i === 1) {
        ctx.drawImage(choki, 250, 100 + 120 * i, 100, 100)
      }
      ctx.drawImage(pa, 350, 100 + 120 * i, 100, 100)
    }
    ctx.fillStyle = 'blue'
    ctx.font = '50pt MS ゴシック'
    ctx.textAlign = 'left'
    ctx.textBaseline = 'top'
    ctx.fillText('ジャンケンゲーム!', 50, 30)
    ctx.font = '25pt MS ゴシック'
    ctx.fillText('エンターキーをおしてね!', 100, 400)
  } else if (mode === 5) {
    ctx.drawImage(haikei, 0, 0, 640, 480)
    ctx.drawImage(janken[computer_hand], 350, 250, 200, 200)
    ctx.drawImage(janken_key, 50, 275, 250, 150)
    if (computer_hand++ >= 2) {
      computer_hand = 0
    }
  }
}
```

### プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_7-1.html]** を読みこみます。
- ② コードの赤字の部分をタイピングして完成後にコードを保存します。
- ③ 〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_7-2.html]** を読みこみます。
- ④ コードの赤字の部分をタイピングして完成後にコードを保存します。
- ⑤ 保存したファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。



“でなければ～”

スクラッチ  
Scratch では右記のブロックを  
つか  
使っていましたが、JavaScript  
では **[else if]** を使います。



タイトル画面でエンターキーをおすとプレイ画面が表示され、  
グーチョキパーの画像が順番にくり返し表示されたかな？



## 8

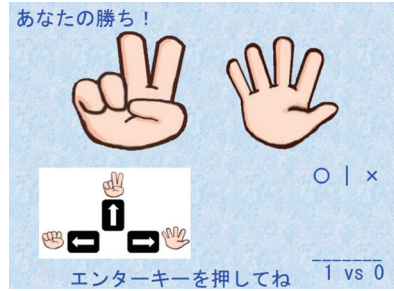
## 結果画面を表示しよう



## 矢印キーで結果画面に切りかえよう

どの矢印キーを選んでも、結果画面に行くようにしましょう。

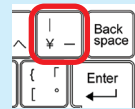
## 表示画面



## 入力するテキスト

```
// キーがおされた時にやること
function keyDown(evt) {
  if (mode === 5) {
    if (evt.keyCode === 37 || evt.keyCode === 38 || evt.keyCode === 39) {
      player_hand = evt.keyCode - 37
      janken_hantei()
      mode = 6
    }
  }
}
```

|| は「または」という意味です。  
通常、キーボードの右上にあります。  
Shift キーをおしながら入力しましょう。



「janken\_hantei()」の function は、じゃんけんの勝ち負けをはん定するプログラムです。  
今回はあらかじめ作ってあるので、よび出しましょう。



## 結果画面を表示しよう

## 入力するテキスト

```
// 画面をえがく
function draw() {
  :
  } else if (mode === 5) {
    ctx.drawImage(haikei, 0, 0, 640, 480)
    ctx.drawImage(janken[computer_hand], 350, 250, 200, 200)
    ctx.drawImage(janken_key, 50, 275, 250, 150)
    if (computer_hand++ >= 2) {
      computer_hand = 0
    }
  } else if (mode === 6) {
    janken_draw_kekka()
  }
}
```



「janken\_draw\_kekka ()」の function は、  
じゃんけんの結果を表示するプログラムです。  
今回はあらかじめ作ってあるので、よび出しましょう。

## プログラミングの手順

- ① <コード写経> のお手本エリアに **[index\_お手本\_ステップ\_8-1.html]** を読みこみます。
- ② コードの赤字の部分をタイピングして完成後にコードを保存します。
- ③ <コード写経> のお手本エリアに **[index\_お手本\_ステップ\_8-2.html]** を読みこみます。
- ④ コードの赤字の部分をタイピングして完成後にコードを保存します。
- ⑤ 保存したファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

プレイ画面で矢印キーをおすと結果画面が表示され、自分の出した  
手とコンピュータの出した手、ジャンケンの結果が表示されたかな？



# 9

## さいしゅうけつ か が めん ひょうじ 最終結果画面を表示しよう



### 3 かいしょう ぶ お さいしゅうけつ か が めん き か 3 回勝負が終わったら最終結果画面に切り替えよう

どの矢印キーを選んでも、結果画面に行くようにしましょう。

#### にゅうりょく 入力するテキスト

```
// キーをおし終えた時にやること
function keyUp(evt) {
  if (mode === 0) {
    if (evt.keyCode === 13) {
      mode = 5
    }
  } else if (mode === 6) {
    if (evt.keyCode === 13) {
      if (shohai.length >= 3) {
        mode = 9
      } else {
        mode = 5
      }
    }
  }
}
```

「shohai.length」で  
じゃんけんの何回勝負目  
かを判定しています。  
もし3回目以上なら、  
mode=9 の分岐に進み  
ます。

#### ひょうじ が めん 表示画面



または



### さいしゅうけつ か が めん ひょうじ 最終結果画面を表示しよう

しょうはいけつ か おう が ぞう ひょうじ  
勝敗結果に応じた画像が表示されるようにします。

#### にゅうりょく 入力するテキスト

```
// 画面をえがく
function draw() {
  :
} else if (mode === 6) {
  janken_draw_kikka()
} else if (mode === 9) {
  if (player_point > computer_point) {
    ctx.drawImage(you_win, 0, 0, 640, 480)
  } else {
    ctx.drawImage(you_lose, 0, 0, 640, 480)
  }
}
}
```

#### てしゅん プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_9-1.html]** を読みこみます。
- ② コードの赤字の部分をタイピングして完成後にコードを保ぞんします。
- ③ 〈コード写経〉のお手本エリアに **[index\_お手本\_ステップ\_9-2.html]** を読みこみます。
- ④ コードの赤字の部分をタイピングして完成後にコードを保ぞんします。
- ⑤ 保ぞんしたファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

- ・3 かいしょう ぶ あと さいしゅうけつ か が めん ひょうじ  
3 回勝負した後、最終結果画面が表示されたかな？
- ・自分が 2 かい いじょう か  
自分が 2 回以上勝ったときは勝ちの絵、コンピュータが 2 回以上  
勝ったときは負けの絵が表示されたかな？



## 3

かいぞう  
改造しよう

## 1

かいしょう ぶ  
5回勝負にしてみよう

プログラムを<sup>へんこう</sup>変更する場所を<sup>かんが</sup>えよう

3回<sup>かいしょう ぶ</sup>勝負のジャンケンゲームを、5回<sup>かいしょう ぶ</sup>勝負に<sup>かいぞう</sup>改造します。どこを<sup>か</sup>変えるかを<sup>かんが</sup>えてみましょう。

ひょうじ が めん  
表示画面

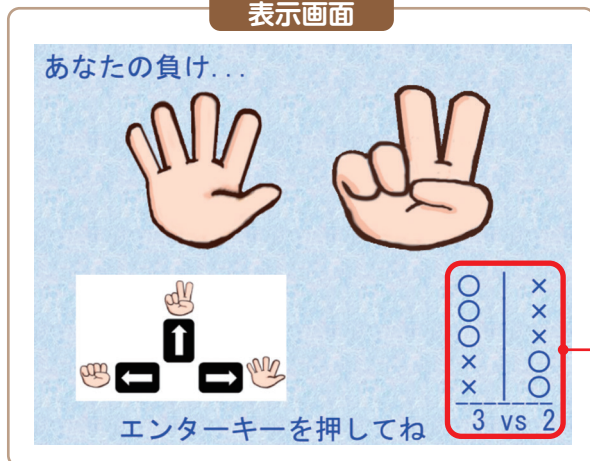
かいぞう  
改造のヒント

ジャンケンの<sup>しょう ぶ</sup>勝負が終わったら<sup>けっ か が めん ひょうじ</sup>結果画面が表示され、<sup>エンター</sup>Enter キーを押すと次の勝負が始まりますが、3回<sup>かいしょう ぶ</sup>勝負が終わっていたら<sup>さいしゅう けっ か が めん ひょうじ</sup>最終結果画面が表示されます。5回<sup>かいしょう ぶ</sup>勝負にするためには、ここを「5回<sup>かいしょう ぶ</sup>勝負が終わったら」<sup>さいしゅう が めん ひょうじ</sup>最終画面が表示されるように<sup>へんこう</sup>変更する<sup>ひつよう</sup>必要があります。次のページの改造後のプログラムを見る前に、これまでのコードを見直して、どこを<sup>へんこう</sup>変えるかを<sup>かんが</sup>えてみましょう。



## どこを改造すればいいか考えよう

### 表示画面



5 回勝負になるようにするには、  
どうすればいいでしょう？  
今、3 回勝負になるように書いてあるところをプログラムの中からさがしてみましょう。

### 入力するテキスト

```
// キーをおし終えた時にやること
function keyUp(evt) {
  if (mode === 0) {
    if (evt.keyCode === 13) {
      mode = 5
    }
  } else if (mode === 6) {
    if (evt.keyCode === 13) {
      if (shohai.length >= 5) {
        mode = 9
      } else {
        mode = 5
      }
    }
  }
}
```

### プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_お手本\_改造\_1.html]** を読みこみます。
- ② コードの赤字の部分をタイピングして完成後にコードを保存します。
- ③ 保存したファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

5 回勝負したあと、最終結果画面が表示されたかな？

☐




## 2 コンピュータを強くしてみよう



### プログラムを変更する場所を考えよう

コンピュータは「グー」「チョキ」「パー」の決まった順番で手を考えているので、プレイヤーがタイミングよく手を出すとコンピュータに勝つことができます。コンピュータが出す手をランダムにすることで、コンピュータが強くなるように改造してみよう。

### JavaScript でのランダムの作り方

Scratch では○から△までの乱数というようにランダムな数を作りました。JavaScript では、0 から 1 までの小数ランダムに発生する機能が用意されています。今回のゲームでは 0 が「グー」、1 が「チョキ」、2 が「パー」になっています。0 から 2 までの整数の乱数を作りたい場合は下のようない命令を使うことができます。

```
Math.floor(Math.random() * 3)
```



### 改造のヒント

コンピュータの出す手は 0 が「グー」、1 が「チョキ」、2 が「パー」の手となるように変数を設定しています。コンピュータの手を設定している部分をプログラムの中から見つけて、上の「JavaScript でのランダムの作り方」で説明した乱数となるように改造してみよう。



どこを改造すればいいか考えよう

入力するテキスト

```
// 画面をえがく
function draw() {
  if (mode === 0) {
    ctx.drawImage(haikei, 0, 0, 640, 480)
    for (let i = 0; i < 3; i++) {
      ctx.drawImage(gu, 150, 100 + 120 * i, 100, 100)
      if (i === 1) {
        ctx.drawImage(choki, 250, 100 + 120 * i, 100, 100)
      }
      ctx.drawImage(pa, 350, 100 + 120 * i, 100, 100)
    }
    ctx.fillStyle = 'blue'
    ctx.font = '50pt MS ゴシック'
    ctx.fillText('ジャンケンゲーム!', 50, 80)
    ctx.font = '25pt MS ゴシック'
    ctx.fillText('エンターキーを押してね!', 100, 450)
  } else if (mode === 5) {
    ctx.drawImage(haikei, 0, 0, 640, 480)
    ctx.drawImage(janken[computer_hand], 350, 250, 200, 200)
    ctx.drawImage(janken_key, 50, 275, 250, 150)
    if (computer_hand++ >= 2) {
      computer_hand = 0
    }
    computer_hand = Math.floor(Math.random() * 3)
  }
}
```

アドバンスコースでは、  
緑の字は削除するコードです。  
お手本を読みこんだら、緑の字  
は消して、赤の字を書いてみま  
しょう。

プログラミングの手順

- ① 〈コード写経〉のお手本エリアに **[index\_お手本\_改造\_2.html]** を読みこみます。
- ② コードの赤字の部分の部分をタイピングして完成後にコードを保存します。
- ③ 保存したファイルを Chrome ブラウザにドラッグ&ドロップします。⇒プログラムが実行されます。

コンピュータの手がランダムに表示されるようになったかな？

☐


以上でスタートアップの説明はおしまいです。  
次月からはアドバンスコースのゲームをテキスト  
を見ながら作って行きましょう！

